

Module 2 Activity 4: Working with Classes

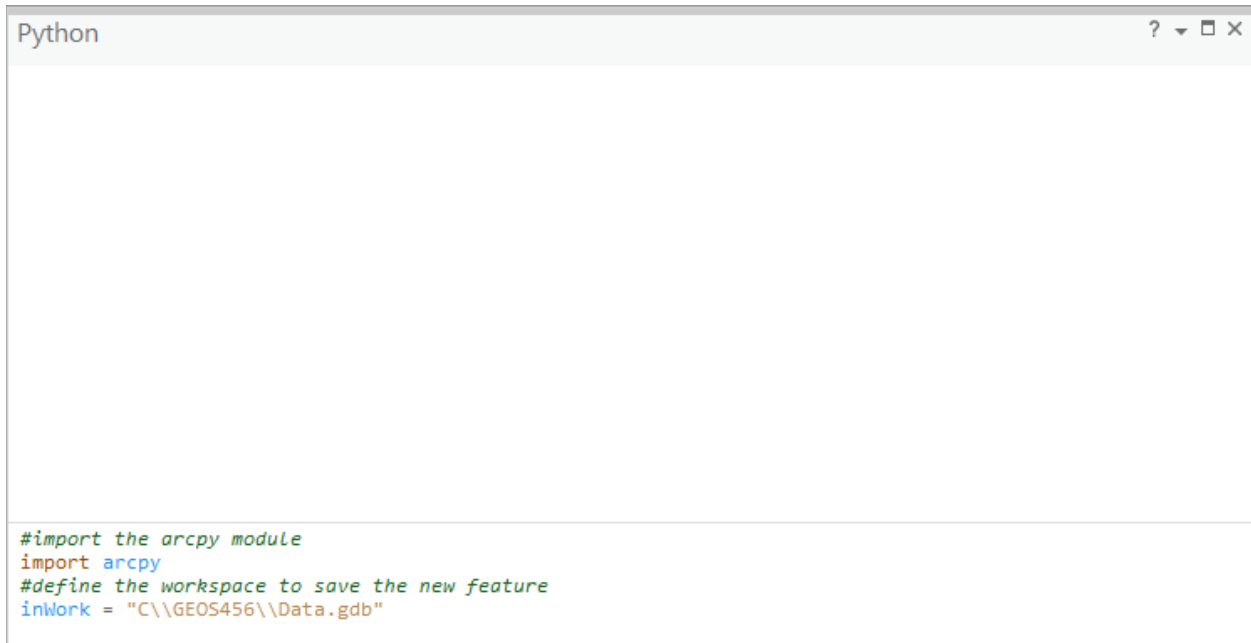
Purpose:

Create a new empty feature class with the same projection as an existing .prj file.

Instructions:

1. Import ArcPy
2. Define the input workspace by creating a variable to hold the pathname value

NOTE: Defining the input workspace is not the same as setting the workspace environment. In this exercise, it may not be the best approach to set the workspace environment as the .prj file might be in a different location than where the output will go.



```
Python
#import the arcpy module
import arcpy
#define the workspace to save the new feature
inWork = "C:\\GEOS456\\Data.gdb"
```

3. Create a variable called **outName**
4. Define the **outName** value with **Parcels**. This will be the name of the new feature class.

NOTE: If you were working with shapefiles instead of a geodatabase feature class, you would need to specify the file extension of the feature with .shp

```
Python
#import the arcpy module
import arcpy
#define the workspace to save the new feature
inWork = "C:\\GEOS456\\Data.gdb"
#assign the name of the new feature to a variable
outName = "Parcels"
```

5. Specify the location of the .prj file to be used to create the new feature class
6. Use the **SpatialReference** class to create an object used to define the output coordinate system of the new feature class

```
Python
#import the arcpy module
import arcpy
#define the workspace to save the new feature
inWork = "C:\\GEOS456\\Data.gdb"
#assign the name of the new feature to a variable
outName = "Parcels"
#identify the location of the ".prj" file
prjFile = "C:\\GEOS456\\prj_files\\Towns.prj"
#use the SpatialReference class to assign the projection file to the new feature
spatialRef = arcpy.SpatialReference(prjFile)
```

In the above screenshot, note that the .prj file defined in **step 5** is referenced in **step 6**

7. Use the **CreateFeatureclass_management()** function to create an empty feature class
8. Specify the new feature class to be a **Polygon** geometry type

```
Python
#import the arcpy module
import arcpy
#define the workspace to save the new feature
inWork = "C:\\GEOS456\\Data.gdb"
#assign the name of the new feature to a variable
outName = "Parcels"
#identify the location of the ".prj" file
prjFile = "C:\\GEOS456\\prj_files\\Towns.prj"
#use the SpatialReference class to assign the projection file to the new feature
spatialRef = arcpy.SpatialReference(prjFile)
#create a new empty feature class with POLYGON geometry
arcpy.management.CreateFeatureclass(inWork, outName, 'POLYGON', spatial_reference = spatialRef)
```

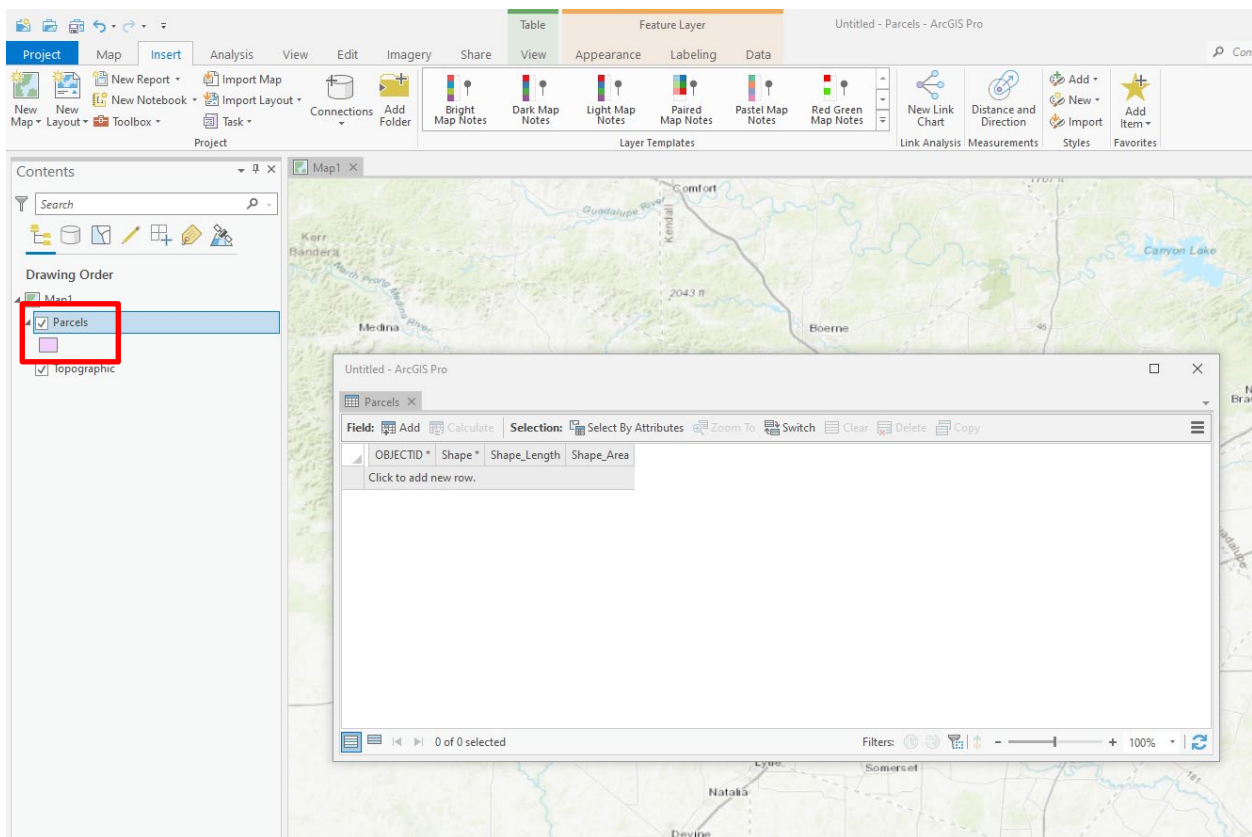
The syntax for the **CreateFeatureclass_management()** function is as follows:

CreateFeatureclass_management (out_path, out_name, {geometry_type}, {template}, {has_m}, {has_z}, {spatial_reference}, {config_keyword}, {spatial_grid_1}, {spatial_grid_2}, {spatial_grid_3})

Note that in the above screenshot, we did not ignore or pass over some of the optional parameters. The reason that empty quotes were not used to skip over {template}, {has_m}, {has_z} is because we specified the **spatial_reference** parameter with an actual value. This process can be done with any tool syntax.

9. Press **enter** twice to run the script

A new empty feature class will be created in your **Data.gdb** and added to Pro.



10. Check the properties of the feature class and review the coordinate system

Layer Properties: Parcels

General
Metadata
Source
Elevation
Selection
Display
Cache
Definition Query
Time
Range
Indexes
Joins
Relates
Page Query

> Extent

Projected Coordinate System
NAD 1983 UTM Zone 12N

Projection
Transverse Mercator

WKID
26912

Authority
EPSG

Linear Unit
Meters (1.0)

False Easting
500000.0

False Northing
0.0

Central Meridian
-111.0

Scale Factor
0.9996

Latitude Of Origin
0.0

Geographic Coordinate System
NAD 1983

WKID
4269

OK
Cancel